Script oral

Le Solitaire · Mattias Mathevon et Jules Damour · 10 minutes sur le dossier, puis 10 minutes de questions

À deux : Mattias parle 5 min 05, Jules 4 min 55. Le texte dit toujours « on » : le passage de l'un à l'autre se fait sans annonce.

Seul : lis tout, et commence par « Je suis Mattias Mathevon, j'ai fait ce projet en binôme avec Jules Damour. »

En retard ? Saute les passages soulignés en pointillés : ils font gagner environ 30 secondes.

00:00 **Ouverture** Mattias 25 s

Bonjour. On est Mattias Mathevon et Jules Damour, et on va vous présenter notre solitaire en mode texte, écrit en Python. On suit notre dossier de deux pages dans l'ordre : ses cinq rubriques, puis une démonstration. Le sujet interdit les classes, donc tout le jeu tient dans des textes, des listes, des dictionnaires et des tuples.

Geste : donner le dossier papier aux profs avant de commencer.

00:25 **Rubrique 1 : les structures de données** Mattias 2 min 30

Une carte, c'est un simple texte : sa valeur, puis sa famille en deux lettres. `"5ca"`, c'est le 5 de carreau. Pour la lire, on coupe par la fin : `carte[:-2]` donne la valeur et `carte[-2:]` donne la famille. On coupe par la fin parce que la famille fait toujours deux lettres, alors que la valeur peut en faire deux, comme pour le 10.

Toutes les cartes sont rangées dans un seul dictionnaire, `piles`, qui contient 13 listes : la pioche `"p"`, la défausse `"d"`, les colonnes `"c1"` à `"c7"` et les quatre fondations. La règle est la même partout : la dernière carte de la liste est celle du dessus. On la lit avec `[-1]`, on la retire avec `pop`, on en pose une avec `append`. Pioche, défausse, colonnes et fondations ont donc la même structure, et le code reste simple.

Il reste deux dictionnaires. `cachees` donne, pour chaque colonne, le nombre de cartes face cachée. Elles sont toujours en bas, donc un seul nombre suffit : une carte est cachée si sa position est plus petite que ce nombre. `index` donne, pour chaque carte, le nom de sa pile. Quand le joueur tape `5ca-c4`, on trouve le départ tout de suite, sans chercher dans les 13 piles. En contrepartie, il faut le mettre à jour à chaque déplacement et à chaque pioche.

Voici ce coup pas à pas. `verif_coup` renvoie le tuple `("5ca", "c4")`. `index` dit que la carte est en c6, et sa position est 5. Le morceau à déplacer est `piles["c6"][5:]` : la carte et tout ce qui est posé dessus. Après les vérifications, `del` retire le morceau, `extend` le pose sur c4 et `index` passe à c4. Il ne reste que des cartes cachées en c6 : `cachees["c6"]` passe de 5 à 4, et le 3 de cœur se retourne.

Montrer : dossier page 1, la liste à gauche et le plateau à droite, puis le paragraphe « Un coup pas à pas ». **Code** : `piles` ligne 23, `cachees` 55, `index` 63, le déplacement 824.

02:55 **Rubrique 2 : les commandes et les menus** Jules 1 min 25

Au lancement, le menu propose cinq choix : nouvelle partie, reprendre la sauvegarde, choisir le niveau, quitter, ou la partie de démonstration, qui a toujours la même donne. Il redemande tant que la réponse n'est pas entre 1 et 5.

Pendant la partie, tout se fait en une seule saisie. Pour déplacer, on tape toujours carte, tiret, pile, même pour plusieurs cartes : on donne la première du groupe, les autres suivent. À côté, il y a `p` pour piocher, `h` pour voir les coups, `ia` pour que l'ordinateur joue, `annuler`, `sauvegarde` et le point d'interrogation pour les règles.

Toutes les saisies sont vérifiées, dans l'ordre du deuxième tableau. `verif_coup` contrôle la forme : un seul tiret, une carte et une pile qui existent. `verif_carte` regarde si la carte a le droit de bouger : ni dans la pioche, ni face cachée. Ensuite, selon l'arrivée, `verif_colonne` contrôle la couleur différente et la valeur juste en dessous, et `verifier_fondation` contrôle la famille et la valeur juste au dessus. Dès qu'une vérification échoue, le jeu dit pourquoi, rien ne bouge et le coup n'est pas compté.

Montrer : dossier page 1, les deux tableaux. **Code** : `menu` ligne 312, la saisie 764, `verif_coup` 403, `verif_carte` 437, `verif_colonne` 477, `verifier_fondation` 541.

04:20 **Rubrique 3 : trouver et classer les coups** Jules 1 min 20

Page 2 : comment l'ordinateur propose les coups. `trouver_coups` ne devine rien. Elle essaie chaque carte, sauf celles de la pioche, sur chacune des 11 piles d'arrivée, soit 572 essais au maximum. Elle garde les coups que le jeu accepterait, en appelant les mêmes fonctions de vérification que pour le joueur, avec `parler=False` pour ne pas afficher les messages de refus. Ce choix a deux avantages : un coup proposé est forcément accepté, et si on corrige une règle, l'ordinateur suit tout seul.

Ensuite, on classe. Chaque coup va dans une liste selon son type, puis on colle les listes avec un plus, dans l'ordre du tableau. D'abord une carte qui monte sur sa fondation, puis un coup qui retourne une carte cachée, une carte de la défausse vers une colonne, une carte qui en libère une autre pour sa fondation, une colonne qu'on vide pour un roi qui attend, et piocher. Tout à la fin, les autres changements de colonne et les cartes qui redescendent d'une fondation.

Montrer : dossier page 2, le tableau des 8 rangs, rang par rang. **Code** : `trouver_coups` ligne 647, le rangement 704, les listes collées 723.

05:40 **Rubrique 3 : l'ordinateur joue tout seul** Mattias 1 min 10

Pour jouer tout seul, c'est la commande `ia`. La boucle du jeu met `auto` à True. À chaque tour, au lieu de lire le clavier, elle prend le premier coup de la liste et le joue avec la même fonction `deplacement` que le joueur.

Il y a deux protections. Les rangs 7 et 8 ne sont jamais joués, sinon l'ordinateur pourrait déplacer un 7 noir d'un 8 rouge à l'autre sans fin. Et `nb_tours` compte les fois où il retourne la défausse sans rien jouer : à la deuxième, il a vu toute la pioche, donc il s'arrête et rend la main.

Quand plus aucune carte n'est cachée et que la pioche et la défausse sont vides, la partie est forcément gagnée, et l'ordinateur range les cartes tout seul. Il gagne la partie de démonstration aux trois niveaux. Sur 500 parties au hasard par niveau, il en gagne 41 % en facile, 21 % en moyen et 11 % en difficile.

Montrer : dossier page 2, les deux protections et l'encadré Résultats. **Code** : `auto` ligne 945, `nb_tours` 951 à 956, la fin automatique 938, `tout_est_retourne` 592.

06:50 **Rubrique 4 : les points forts** Jules 40 s

Nos points forts. L'ordinateur sait jouer, et il rend la main au lieu de tourner en rond. Un coup se fait toujours en une saisie, et chaque refus est expliqué. On peut annuler le dernier coup, même une pioche ou un coup de l'ordinateur, et sauvegarder pour reprendre au même niveau. Le plateau tient en 18 lignes sur 56 caractères. Et on a une partie gagnante rejouable d'un coup, en 95 coups : on l'a relancée après chaque modification pour vérifier qu'on n'avait rien cassé.

Code : la copie pour annuler ligne 929, `annuler` 991, `sauvegarder` 365, `recharger_partie` 384.

07:30 **Rubrique 5 : les niveaux** Mattias 40 s

Dernière rubrique, les niveaux. Le niveau se choisit au menu. C'est un simple entier, `difficulte`, égal au nombre de cartes tirées à chaque pioche : 1 en facile, 2 en moyen, 3 en difficile. La défausse s'affiche en éventail : on voit les cartes de la dernière pioche, mais seule la dernière est jouable, les autres sont en gris. C'est ce qui rend le niveau difficile plus dur : si on ne joue rien, ce sont toujours les mêmes 8 cartes sur 24 qui arrivent sur le dessus.

Code : `choisir_difficulte` ligne 337, la boucle de `piocher` 619, `afficher_defausse` 168.

08:10 **Démonstration** Mattias tape Jules commente 1 min 30

`python3 Solitaire_Mathevon_Damour.py`, puis `5`	Voici la partie de démonstration : les cartes cachées en gris, les cœurs et les carreaux en rouge.
`6tr-c2`	On essaie de poser le 6 de trèfle sur le 8 de trèfle. Le jeu refuse, dit pourquoi, et rien ne bouge.
`h`, puis Entrée	`h` propose d'abord `5ca-c4`, puis `p` : c'est notre classement.
`5ca-c4`	On le joue : le 5 de carreau part sur le 6 de trèfle, et le 3 de cœur se retourne.
`annuler`	Annuler remet la partie comme avant le coup.
`ia`, puis 30 s de silence	Maintenant, l'ordinateur joue tout seul, avec les mêmes règles que le joueur. [À la fin :] Il gagne en 91 coups.

Vérifié sur le fichier rendu : cet enchaînement finit par « Partie gagnée en 91 coups ! », avec ou sans `annuler`. `ia` met 27 s, plus 3 s d'animation. Le jeu s'arrête après la victoire : garder `ia` pour la fin.

09:40 **Clôture** Mattias 20 s

Pour résumer : toutes les piles ont la même structure, et chaque règle est écrite à un seul endroit, partagé par le joueur et l'ordinateur. Le code rendu tient dans un seul fichier, et on a travaillé à deux avec git. On est prêts pour vos questions.

Les 10 minutes de questions

Devant une demande de modification

1. Reformule la demande, et fais-la préciser si elle est floue.
2. Dis où tu vas modifier et pourquoi, avant de toucher au code.
3. Fais le plus petit changement, dans `Solitaire_Mathevon_Damour.py`.
4. Relance, teste le cas demandé, puis un coup normal.
5. Pense aux effets de bord : `h` et `ia`, l'écran des règles, la partie de démonstration, la sauvegarde, `annuler`.
6. Si ça plante, lis la dernière ligne de l'erreur et son numéro de ligne, puis `Cmd+L` pour y aller.

Où aller dans le fichier unique

Les défis du site citent encore les 14 anciens fichiers. Le code à remplacer se trouve bien dans le fichier unique, sauf une ligne d'import du défi sur la pioche, qui ne sert plus. Les numéros valent tant que le fichier n'a pas changé de nombre de lignes.

Le prof demande	Lignes
Défis faciles	
Toutes les cartes des colonnes visibles au début	`93, et 55 pour la démo`
Majuscules et espaces acceptés dans la saisie	`après 766`
Dos des cartes en `[##]`	`125`
Cœurs et carreaux en jaune	`114`
Une pioche compte comme un coup	`986 à 988`
Figures en majuscules : `[D♥]`	`142`
`h` limité aux 3 meilleurs coups	`739 à 748`
Défis moyens	
Compteur de coups sous le plateau	`juste avant 963`
Une carte ne redescend plus d'une fondation	`459 à 463`
N'importe quelle carte sur une colonne vide	`483 à 490, règles 279`
Même couleur au lieu de couleur alternée	`507 à 515, règles 277`
Commande quitter	`796, 970, 1003, règles 290`
Annuler plusieurs coups de suite	`917, 981, 992, règles 289`
Niveau expert à 4 cartes	`308, 338 à 350, 169, règles 286`
Score : +10 par fondation, -2 par pioche	`921, 963`
Défis vicieux	
Fondations du roi jusqu'à l'as	`551, 565, 936, règles 280`
Pioche retournée 2 fois au plus en difficile	`613, 628`
8 colonnes au lieu de 7	`Cmd+F sur c7, puis sur 7`
Proposer de rejouer après la victoire	`tout le lancement, 879 à 1003`
Détecter une partie perdue	`934, 1003`

Les pièges propres au fichier unique

Ils viennent de la fusion du 4 novembre et ne sont pas sur le site. Un prof qui lit le fichier d'un bloc peut les voir.

Question possible	Lignes	Réponse
`rouge` est défini deux fois, pourquoi ça ne casse rien ?	`114, 480`	La ligne 480 crée une variable locale à `verif_colonne`. Elle masque le code couleur dans cette fonction seulement ; l'affichage lit toujours la globale.
`index` : dictionnaire ou méthode ?	`63, 445`	`index[carte]` lit notre dictionnaire et donne la pile. `piles[depart].index(carte)` est la méthode des listes et donne la position.

Pourquoi `valeur` et `ordre`, deux fois la même liste ?	481, 538	Ça vient de deux fichiers fusionnés. Ça marche, et on pourrait n'en garder qu'une. Le dire franchement.
À quoi sert `move_possible` dans `deplacement` ?	756	À rien : la vraie vérification est dans `verif_coup`, ligne 404. On peut supprimer la ligne 756.
Pourquoi `piles = copie_piles` marche-t-il ici ?	995	Cette ligne est au niveau du fichier, donc elle remplace la variable du jeu. Dans une fonction, elle ne changerait que la variable locale.
Pourquoi pas de `if __name__ == "__main__"` ?	879	Le fichier est un jeu à lancer directement, il n'est importé nulle part. Le lancement est écrit au niveau du fichier.
Ce commentaire sur `[-2:]` est-il juste ?	534	Non : il dit « avant-avant-dernière lettre ». `[-2:]` part de l'avant-dernière lettre, donc donne les 2 dernières.

Avant d'entrer

- Dossier papier imprimé : il se donne en début de soutenance.
- Si une modification tourne mal : `Cmd+Z`, ou `git restore Solitaire_Mathevon_Damour.py` pour revenir au dernier commit.
- Si Jules projette depuis son ordi : le fichier unique est sur la branche `mattias`, donc `git fetch origin` puis `git switch mattias`.
- Les anciens fichiers (`main.py`, `deplacement.py`…) sont encore dans le projet. Modifie et lance seulement `Solitaire_Mathevon_Damour.py`, et vérifie ce que relance `Ctrl+R`.
- Terminal agrandi, police lisible à la projection, au moins 20 lignes sur 60 colonnes. Rejoue la démo une fois.